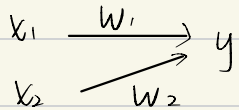
深度学习

感知机

基本结构：



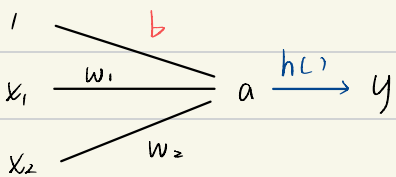
$$y=\begin{cases}0 & (w_1x_1+w_2x_2+b\le 0)\\ 1 & (w_1x_1+w_2x_2+b>0)\end{cases}$$

可以通过调整参数实现逻辑门 但只能划分出线性空间，因此单层感知机不能实现 XOR

定义 $a=b+w_1x_1+w_2x_2$

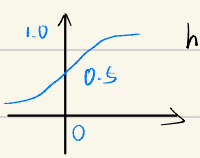
$y=h(a)$，$h(x)=\begin{cases}0 & a\le 0\\ 1 & a>0\end{cases}$

$h(x)$ 为激活函数



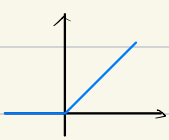
其他激活函数：

① Sigmoid：$h(x)=\frac{1}{1+e^{-x}}$

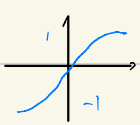


$$h'(x)=h^2(x)e^{-x}$$
$$=h(x)(1-h(x))$$

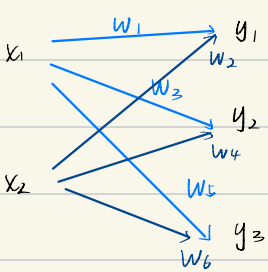
② ReLU：$h(x)=\begin{cases}x & x>0\\ 0 & x\le 0\end{cases}$



② $\tanh(u)=\frac{e^u-e^{-u}}{e^u+e^{-u}}$



矩阵内积表示参数加权和：



$$X \quad dot \quad W \quad = \quad Y$$
$$(x_1, x_2)\begin{pmatrix} w_1 & w_3 & w_5\\ w_2 & w_4 & w_6\end{pmatrix}\quad (y_1, y_2, y_3)$$

输出层：一般要改变输出层的激活函数

① 恒等函数　② Softmax 函数　$y_k = \frac{e^{a_k}}{\sum e^{a_i}} = \frac{Ce^{a_k}}{C\sum e^{a_i}} = \frac{e^{a_k + \log C}}{\sum e^{a_i + \log C}}$

为了防止出现溢出，可以减去 Max a_i，不影响 Softmax 的值

神经网络（多层感知机）的参数结构：（以 MINIST 手写识别为例）

X	W_1	W_2	W_3	Y
×100				×100
28×28=784	784×50	50×100	100×10	10
28×28 pixels	50个神经元	100个神经元	10个类别（0~9）	

批处理
batch_size = 100
epoch = 一个总样本

神经网络

神经网络的学习场景

Samples
- 算法
- 特征量（SIFT、HOG）→ 机器学习（SVM、KNN）
- 神经网络（深度学习）（将数据作为原始数据，进行"端到端"学习）

监督数据的 one-hot 表示：

$y=[0.1,\ 0.05,\ 0.6,\ 0.0\ \ldots]$　　$t=[0,\ 0,\ 1,\ 0,\ 0\ \ldots]$

是"0"、"1"、"2"的概率　　是"2"的概率是1 → 真实标签就是2

损失函数：表示神经网络性能"恶劣程度"，连续变化

① 均方误差：$E=\frac{1}{2}\sum_k (y_k-t_k)^2$　y_k-输出　t_k-真实值　k-维数

② 交叉熵误差：$E=-\sum_k t_k \log y_k$　y_k-输出　t_k-是否是真实值，　k-维数

数值微分：基于一个给定 Δx 的 Δy，$\frac{\Delta y}{\Delta x} \approx y'_x$

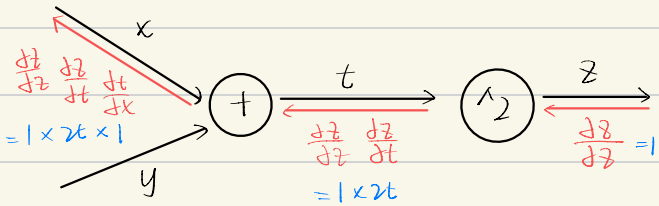
梯度：$\left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \ldots\right)$，由全部偏导数汇总而成的向量（指向各点函数值减小最多方向）

Corner Case：鞍点，梯度小的"学习高原"

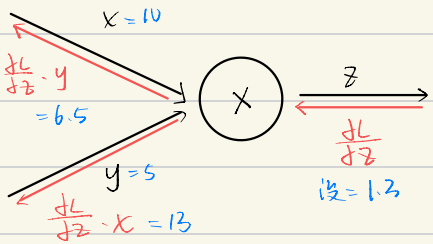
梯度下降法：$x_i = x_i - \eta \frac{\partial f}{\partial x_i}$，　η-学习率

梯度确认：比较数值微分和反向传播算法得到的梯度大小

反向传播



乘法的反向传播：



定义一个计算层：

def forward(self, x, y) → out

def backward(self, dout) → dx, dy

（激活函数可以函数化定义 mask = x<=0）

ReLU层

一般的神经网络结构

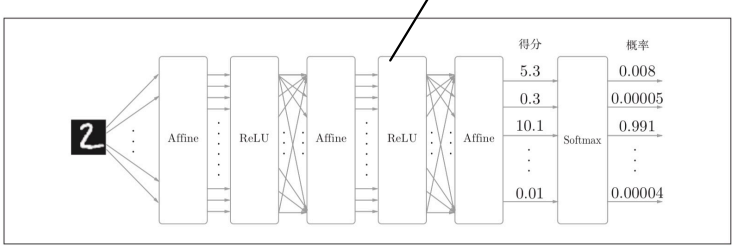
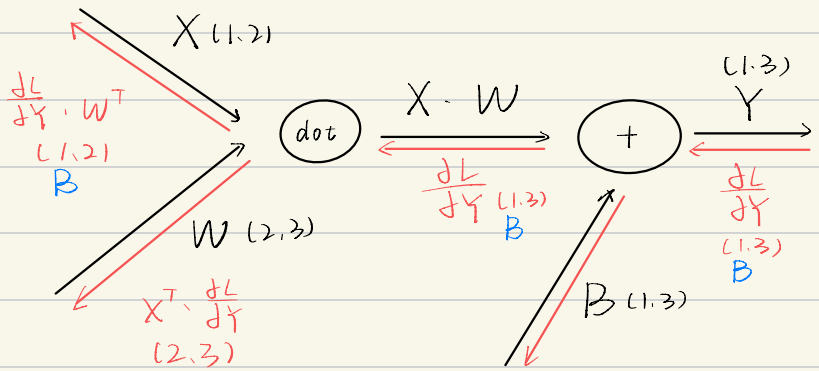


图5-28 输入图像通过Affine层和ReLU层进行转换，10个输入通过Softmax层进行正规化。在这个例子中，“0”的得分是5.3，这个值经过Softmax层转换为0.008（0.8%）；“2”的得分是10.1，被转换为0.991（99.1%）

Affine层（加权求和、偏置） B = Batch_size



Softmax-with-Loss 层

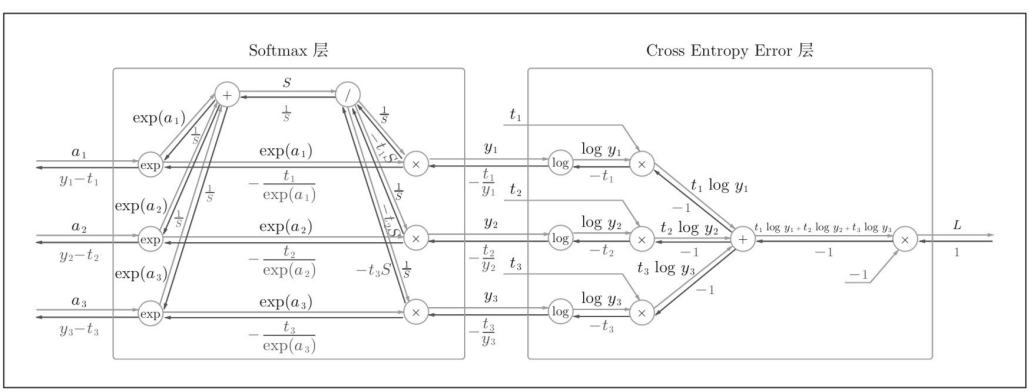


图5-29 Softmax-with-Loss层的计算图

输出 (y_1, y_2, y_3)、监督数据 (t_1, t_2, t_3)、反向传播 $(y_1-t_1, y_2-t_2, y_3-t_3)$

softmax的损失函数 ≤ 分类问题 = Softmax + 交叉熵误差

回归问题 = 恒等函数 + 平方和误差

表5-1　TwoLayerNet类的实例变量

实例变量	说明
params	保存神经网络的参数的字典型变量。 params['W1']是第1层的权重，params['b1']是第1层的偏置。 params['W2']是第2层的权重，params['b2']是第2层的偏置
layers	保存神经网络的层的**有序字典型变量**。 以layers['Affine1']、layers['ReLu1']、layers['Affine2']的形式，通过有序字典保存各个层
lastLayer	神经网络的最后一层。 本例中为SoftmaxWithLoss层

表5-2　TwoLayerNet类的方法

方法	说明
__init__(self, input_size, hidden_size, output_size, weight_init_std)	进行初始化。 参数从头开始依次是输入层的神经元数、隐藏层的神经元数、输出层的神经元数、初始化权重时的高斯分布的规模
predict(self, x)	进行识别(推理)。 参数x是图像数据
loss(self, x, t)	计算损失函数的值。 参数x是图像数据、t是正确解标签
accuracy(self, x, t)	计算识别精度
numerical_gradient(self, x, t)	通过数值微分计算关于权重参数的梯度(同上一章)
gradient(self, x, t)	通过误差反向传播法计算关于权重参数的梯度

参数学习 (将参数学习方法封装成类)

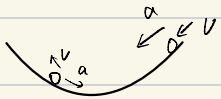
```
class SGD:
    def __init__(self, lr=0.01):
        self.lr = lr

    def update(self, params, grads):
        for key in params.keys():
            params[key] -= self.lr * grads[key]
```

1. SGD stochastic gradient descent 随机梯度下降法

$W \leftarrow W - \eta \frac{\partial L}{\partial W}$ 如果函数的形状非均向(anisotropic),则非常低效

2. Momentum



$v \leftarrow \alpha v - \eta \frac{\partial L}{\partial W}$ v更像物理上的速度, η项更像加速度

$W \leftarrow W + v$ 受步长影响更小,精度更高,收敛更快

3. AdaGrad (Adaptive Grad)

$h \leftarrow h + (\frac{\partial L}{\partial W})^2$ 保存W前所有梯度值的平方和 (+= grads[k] * grads[k])

$W \leftarrow W - \eta \frac{1}{\sqrt{h}} \frac{\partial L}{\partial W}$

可以按参数的元素进行学习率衰减,使变动大的参数的学习率逐渐减小

4. Adam (AdaptiveGrad + Momentum)

$v \leftarrow \alpha v - \beta_1 \frac{\partial L}{\partial W}$

$h \leftarrow h + (\frac{\partial L}{\partial W})^2$

$W \leftarrow W + v - \beta_2 \frac{1}{\sqrt{h}} \frac{\partial L}{\partial W}$

隐藏层的激活值：

多神经元输出相同的值，表现力会受限，相当于1个神经元

故：如果前一层节点数为n，则初始值使用标准差为$\frac{1}{\sqrt{n}}$的分布

Sigmoid：S型，输出→0/1，导数→0，梯度消失，层次不宜太深

ReLU：使用标准差为$\sqrt{\frac{2}{n}}$的高斯分布

Batch Normalization：

以Batch为单位，进行使数据分布的均值为0，方差为1的正规化

正则化：

参数多，表现能力强
训练数据少
> 过拟合

权值衰减：权重取值过大
Dropout：简化形状

权值衰减：在损失函数L中加入$\frac{1}{2}\lambda W^2$（参数的L2范数项）作为正则化项

在反向传播的时候加上正则化项的导数λW

Dropout：正向传播正常传播，反向传播不传播(0)（本质集成学习）

超参数最优化：（贝叶斯优化器，详见《机器学习》中麻将AI优化的部分）

卷积神经网络（详可见《智能计算系统》）

全连接层：忽视数据形状

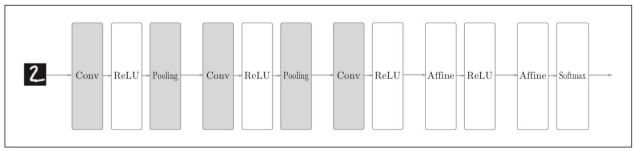


图7-2　基于CNN的网络的例子：新增了Convolution层和Pooling层（用灰色的方块表示）

如图7-2所示，CNN中新增了Convolution层和Pooling层。CNN的层的连接顺序是“Convolution - ReLU -（Pooling）”（Pooling层有时会被省略）。这可以理解为之前的“Affine - ReLU”连接被替换成了“Convolution - ReLU -（Pooling）”连接。

还需要注意的是，在图7-2的CNN中，靠近输出的层中使用了之前的“Affine - ReLU”组合。此外，最后的输出层中使用了之前的“Affine - Softmax”组合。这些都是一般的CNN中比较常见的结构。

滤波器：

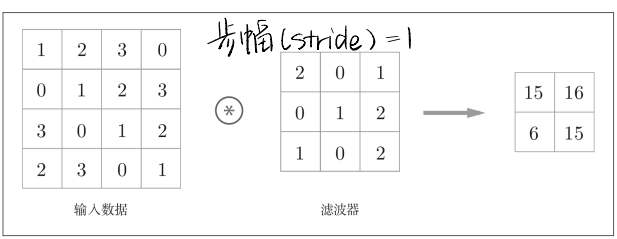


图7-3　卷积运算的例子：用“⊛”符号表示卷积运算

边缘数据要进行填充（Padding）

池化（Pooling）：概括数据

多值滤波器：1值输出

批处理：N值输出

深度学习：层数更深的学习

Data Augmentation：旋转、平移、裁剪、翻转 样本

叠加小型滤波器：减少参数数量，扩大感受野

加深层：减少学习数据，分层次传递信息

受限玻尔兹曼机

Hopfield 神经网络 = ①权重双向对称 ②没有自环 ③随机异步更新

节点状态更新的方法和感知机一致，输入/出 隐藏层一致

$$\text{单元的输出 } X_i(t+1)\begin{cases}1 & u_i(t)>0\\ X_i(t) & u_i(t)=0\\ 0 & u_i(t)<0\end{cases}$$

判别函数 所有相连节点加权和

$$u_i(t)=\sum_{j=1}^{n} w_{ij}x_j(t)-b_i(t)$$

能量函数 $E=\sum_{i=1}^{n} b_i x_i-\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} w_{ij}x_i x_j$（所有节点阈值加权和；所有连接节点权重加权和）

$$\Delta E_k=-\left(\sum_{j=1}^{n} w_{kj}x_j(t+1)-b_k(t+1)\right)\left(x_k(t+1)-x_k(t)\right)=-u_k(t+1)\Delta x_k$$

E是非递增的，随更新减小直至稳定

联想记忆 = 输入某状态x，输出某状态y（一致 = 自联想，E=0，不一致 = 异联想）

应用 = 输入受损图像，恢复原始图像 / 从残缺数据补全完整模式

模式间相似 / 模式复杂 → 串扰（网格单元数 15%）

玻尔兹曼机（训练类似 Hopfield）

$$\begin{cases}P(x_i=1\mid u_i)=\dfrac{e^{\frac{u_i}{T}}}{1+e^{\frac{u_i}{T}}}\\ P(x_i=0\mid u_i)=\dfrac{1}{1+e^{\frac{u_i}{T}}}\end{cases}$$

T = 温度系数，极小值 $=\frac{c}{\log(t+1)}$

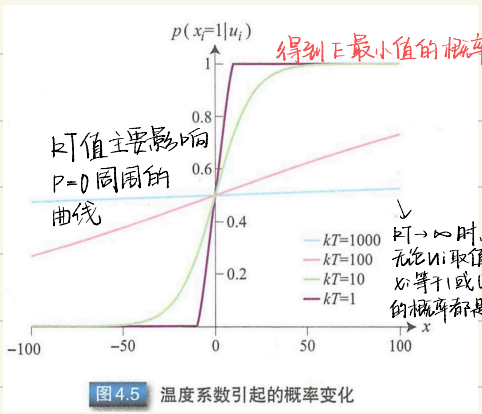


图4.5 温度系数引起的概率变化

得到E最小值的概率高

kT值主要影响P=0周围的曲线

kT→∞时，无论ui取值xi等于1或0的概率都是½

模拟退火 = 先粗后细

跳出局部最优解的概率最高

玻尔兹曼机的参数：

全/任意连接的离散0/1分布（node不是输入层就是隐藏层）

参数：　　状态：$v=\{0,1\}^D$ 可见层　$h=\{0,1\}^P$ 隐藏层

$L=[L_{ij}]_{D\times D}$ 可见层节点间　概率：$P(v,h)=\frac{1}{Z}e^{-E(v,h)}$

$J=[J_{ij}]_{P\times P}$ 隐藏层节点间　能量：$E(v,h)=-(\underbrace{v^TWh}_{v,h\text{间}}+\underbrace{v^TLv/2}_{v\text{内部}}+\underbrace{h^TJh/2}_{h\text{内部}})$

$W=[W_{ij}]_{D\times P}$ v,h节点间

v^TWh：$D\times P$项　v^TLv：$\frac{D^2}{2}$项　h^TJh：$\frac{P^2}{2}$项

$\sum_{i=1}^{D}\sum_{j=1}^{P}v_iW_{ij}h_j$

也可以不乘，因为参数是要学习的，结果上不影响